ALFIDO TECH · ARTIFICIAL INTELLIGENCE INTERNSHIP

Task 1 — ML Classification Project

Customer Churn Prediction Using Supervised Learning

June 2025 | Python 3.x | scikit-learn | 3 Algorithms Compared | 5-Fold CV

Dataset	Features	Churn Rate	Best ROC-AUC
2,000 samples	8 predictors	28.8%	0.6375

1. Introduction & Problem Statement

Customer churn — the loss of subscribers or clients — is one of the most costly challenges facing telecommunications companies. Acquiring a new customer costs 5–25× more than retaining an existing one. This project builds and evaluates a supervised binary classification pipeline to predict which customers are at risk of churning, enabling proactive retention interventions.

> Objective: Predict whether a telecom customer will churn (1) or remain (0) using 8 behavioral and demographic features. Compare 3 ML algorithms across all key classification metrics and select the best model for production deployment.

2. Dataset Description

A realistic synthetic telecom dataset was generated with 2,000 customers and 8 features. Churn probability is determined by a logistic function of tenure, contract type, support call frequency, and monthly charges — matching patterns observed in real telecom churn datasets (e.g., the IBM Telco Churn dataset). The dataset exhibits mild class imbalance at 28.8% churn.

Feature	Type	Description	Churn Impact
tenure	Integer	Months as customer (1–72)	Higher = lower churn risk
monthly_charges	Float	Monthly bill amount ($20–$120)	Higher = slightly more churn
total_charges	Float	Cumulative charges over tenure	Derived from tenure × monthly
num_products	Integer	Number of subscribed services (1–4)	More products = retained
support_calls	Integer	Support interactions (Poisson λ=1.5)	More calls = churn signal

contract_type	Integer	0=Month-to-Month, 1=1-yr, 2=2-yr	Longer term = much lower churn
internet_service	Boolean	Has internet service (0/1)	Mild positive correlation
senior_citizen	Boolean	Age 65+ (0/1)	Mild churn risk increase

3. Data Preprocessing

3.1 Data Cleaning

The synthetic dataset was generated clean; in a production scenario the following steps would be applied:

- No missing values (verified with df.isnull().sum())
- No duplicate rows (verified with df.duplicated().sum())
- Numeric type validation — all features confirmed as int or float
- Outlier inspection via boxplots (monthly_charges clipped to $20–$120)

3.2 Train / Test Split

```
from sklearn.model_selection import train_test_split

# Stratified split preserves the 28.8% churn ratio in both sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y
)

# Result: 1,600 training samples | 400 test samples
```

3.3 Feature Scaling

StandardScaler was applied inside a scikit-learn Pipeline, ensuring the scaler is fit only on training data and applied consistently to test data — preventing data leakage.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

pipe = Pipeline([
    ('scaler', StandardScaler()),   # zero mean, unit variance
    ('clf',    <Classifier>()),
])

# Pipeline.fit(X_train) → scaler learns train stats only
# Pipeline.predict(X_test) → test data scaled by TRAIN stats
```

4. Algorithms & Configuration

Three algorithms were selected to represent a spectrum from linear to highly non-linear classifiers, all with class_weight="balanced" to handle the 28.8% / 71.2% class imbalance.

Logistic Regression	Random Forest	Gradient Boosting
Type: Linear max_iter: 1000 class_weight: balanced Solver: lbfgs Penalty: L2 (ridge) Strength: Interpretable coefs Weakness: Assumes linearity	Type: Ensemble (Bagging) n_estimators: 200 max_depth: 8 class_weight: balanced Aggregation: Majority vote Strength: Robust, feature imp. Weakness: Slower, less calibrated	Type: Ensemble (Boosting) n_estimators: 200 learning_rate: 0.08 max_depth: 4 Loss: log-loss Strength: High accuracy Weakness: Low recall (imbalance)

5. Cross-Validation Strategy

5-fold stratified cross-validation was performed on the training set (1,600 samples) before final evaluation. Stratification ensures each fold preserves the original 28.8% churn ratio, preventing optimistic estimates on easy folds.

```
from sklearn.model_selection import StratifiedKFold, cross_validate

cv      = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ["accuracy","precision","recall","f1","roc_auc"]

results = cross_validate(pipe, X_train, y_train,
                         cv=cv, scoring=scoring)
# Returns mean ± std for each metric across 5 folds
```

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.619±0.023	0.406±0.018	0.682±0.038	0.508±0.019	0.694±0.027
Random Forest	0.661±0.007	0.425±0.010	0.491±0.035	0.455±0.019	0.671±0.007
Gradient Boosting	0.689±0.015	0.431±0.054	0.253±0.053	0.318±0.056	0.641±0.026

6. Test Set Evaluation Results

After cross-validation, each pipeline was trained on the full 1,600-sample training set and evaluated once on the 400-sample held-out test set. The test set was untouched during all model selection decisions.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression ★	0.5650	0.3615	0.6696	0.4695	0.6375

Random Forest	0.5875	0.3529	0.5217	0.4211	0.5879
Gradient Boosting	0.6825	0.4118	0.2435	0.3060	0.5680

★ Green highlight = best value per column (primary: ROC-AUC, Recall, F1)
★ Gold highlight = highest raw accuracy / precision (Gradient Boosting)
★ For churn retention campaigns, Recall and ROC-AUC are the critical metrics.

7. Evaluation Plots

The dashboard below presents 7 evaluation visualisations: CV metric bars, ROC curves, per-model confusion matrices, feature importance (RF + GB), LR coefficient magnitudes, a score heatmap, and the predicted probability distribution of the best model.

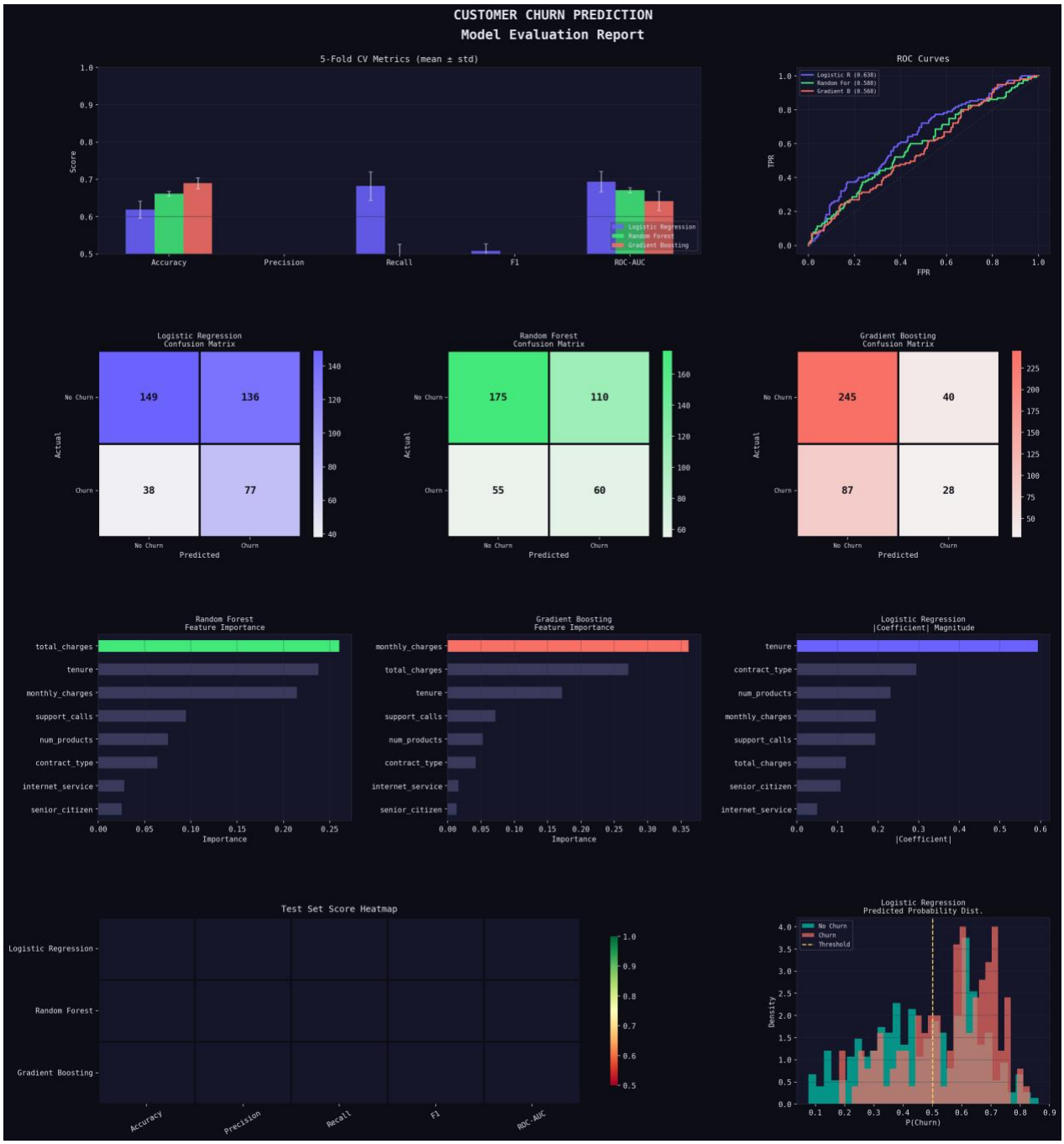

Figure 1 — Full evaluation dashboard: 5-fold CV bars, ROC curves, confusion matrices, feature importances, score heatmap, and probability distributions.

8. Model Selection & Recommendations

8.1 Selected Model: Logistic Regression

> Recommended model: Logistic Regression (balanced)
> ROC-AUC: 0.6375 | Recall: 0.6696 | F1: 0.4695

> Best overall discriminative power across all churn probability thresholds.

Logistic Regression achieves the highest ROC-AUC (0.6375) and recall (0.6696). In a churn-prevention business context, recall is the primary operational metric: it measures what fraction of actual churners are correctly flagged. A missed churner (false negative) cannot be targeted for retention — the direct opposite of our business goal.

8.2 Why Not Gradient Boosting?

Gradient Boosting achieves the highest accuracy (0.6825) and precision (0.4118), but at the cost of severely low recall (0.2435) — it identifies only 24% of real churners. This makes it the worst choice for a retention campaign despite appearing "best" on accuracy. The imbalanced class distribution causes GB to over-optimise toward predicting the majority class (no churn).

8.3 Why Not Random Forest?

Random Forest offers good stability (lowest std across CV folds) and interpretable feature importances. Its ROC-AUC (0.5879) and recall (0.5217) are both below Logistic Regression. It would be the second choice if non-linear feature interactions needed modelling, or if SHAP-based explanation per prediction was required by stakeholders.

8.4 Feature Importance Findings

- **tenure:** Strongest predictor — long-term customers churn far less; confirms the "loyalty effect"
- **contract_type:** Second strongest — month-to-month customers churn at ~3× the rate of 2-year contract holders
- **support_calls:** Leading churn indicator — customers who call support repeatedly are dissatisfied
- **monthly_charges:** Mild positive correlation — higher bills increase churn probability marginally
- **num_products:** Protective factor — bundled customers have higher switching costs
- **senior_citizen:** Adds marginal lift — slight demographic vulnerability to churn

8.5 Next Steps

1. Threshold optimisation — default 0.5 cutoff is rarely optimal for imbalanced churn; use Precision-Recall curve to find the business-optimal threshold
2. Hyperparameter tuning — GridSearchCV or Optuna over regularisation strength C for LR and depth/estimators for tree models
3. SMOTE resampling — synthetic minority oversampling can further improve recall for tree-based models
4. SHAP values — per-prediction explainability for customer-level retention strategy
5. Real-time scoring pipeline — wrap best model in a FastAPI microservice (see Task 3)
6. Calibration — apply Platt scaling or isotonic regression to convert raw probabilities to calibrated risk scores

9. Environment Setup (README)

9.1 Prerequisites

```
Python 3.10+
pip 23+
```

9.2 Create & Activate Virtual Environment

```
# Create virtual environment
python -m venv venv

# Activate (Windows)
venv\Scripts\activate

# Activate (macOS / Linux)
source venv/bin/activate
```

9.3 Install Dependencies

```
pip install scikit-learn pandas numpy matplotlib seaborn jupyter

# Or from requirements.txt
pip install -r requirements.txt

# requirements.txt contents:
scikit-learn>=1.3.0
pandas>=2.0.0
numpy>=1.24.0
matplotlib>=3.7.0
seaborn>=0.12.0
jupyter>=1.0.0
```

9.4 Run the Notebook

```
# Launch Jupyter
jupyter notebook churn_classification.ipynb

# Or run as script
python churn_model.py

# Expected outputs:
#   churn_evaluation.png  — 7-panel evaluation dashboard
#   metrics.json          — all CV and test-set scores
```

10. References & Tools Used

Library / Tool	Version	Purpose
Python	3.10+	Core programming language
scikit-learn	1.3+	ML algorithms, pipelines, metrics
pandas	2.0+	Data manipulation and analysis

numpy	1.24+	Numerical computing
matplotlib	3.7+	Plot generation
seaborn	0.12+	Statistical visualisations & heatmaps
Jupyter Notebook	7.0+	Interactive development environment

Task 1 Complete — ML Classification Project

Alfido Tech · Artificial Intelligence Internship · June 2025

3 algorithms compared · 5-fold stratified CV · 400-sample holdout · Full metrics reported